

SQL Transactions

1. What is a Transaction?

Definition

A **Transaction** is a sequence of one or more SQL operations (queries, inserts, updates, deletes) treated as a single logical unit of work. It ensures database integrity and consistency.

Key properties of transactions (ACID)

- **Atomicity:** All operations succeed or none do.
- **Consistency:** Transaction brings the database from one valid state to another.
- **Isolation:** Concurrent transactions do not interfere.
- **Durability:** Once committed, changes persist even on failure.

2. Unit of Work

Unit of Work

A transaction consists of a sequence of query and/or update statements. According to the SQL standard:

- A transaction begins **implicitly** when an SQL statement is executed.
- The transaction ends when one of the following statements is executed:
 - **COMMIT [WORK]:** Commits the current transaction, making all changes permanent. Automatically starts a new transaction afterward.
 - **ROLLBACK [WORK]:** Rolls back (undoes) all updates made by the transaction, restoring the database to the state before the first statement of the transaction.
- The keyword **WORK** is optional in both **COMMIT** and **ROLLBACK**.

3. Transaction States

States of a Transaction

A transaction goes through the following states:

1. **Active:** Transaction is executing.
2. **Partially Committed:** Last statement executed, waiting to commit.
3. **Committed:** Changes are permanently recorded.
4. **Failed:** Transaction has encountered an error.
5. **Aborted:** Changes are rolled back; database restored to original state.

4. Transaction Control Commands

Basic Commands

- **BEGIN TRANSACTION** or **START TRANSACTION:** Begin a new transaction.
- **COMMIT:** Permanently save all changes made during the transaction.
- **ROLLBACK:** Undo all changes since the start of the transaction or the last savepoint.
- **SAVEPOINT savepoint_name:** Create a named point to which you can rollback partially.
- **RELEASE SAVEPOINT savepoint_name:** Remove a savepoint.

Example Transaction

```
START TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 100 WHERE account_id = 101;  
UPDATE accounts SET balance = balance + 100 WHERE account_id = 202;
```

```
COMMIT;
```

5. Importance of Transactions

Why use transactions?

- To ensure **data integrity** during concurrent access.
- To prevent **partial updates** that can corrupt data.
- To support **error recovery** by rolling back failed operations.
- To maintain **consistency** of the database.

6. Isolation Levels

Definition

Isolation levels control how/when the changes made by one transaction become visible to others.

Isolation Level	Description
Read Uncommitted	Transactions may see uncommitted changes (dirty reads).
Read Committed	Only committed data is visible; prevents dirty reads.
Repeatable Read	Prevents non-repeatable reads by locking data read by the transaction.
Serializable	Highest isolation; transactions are completely isolated (serial execution).

Set Isolation Level in SQL

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

7. Transaction Anomalies

Common Anomalies

- **Dirty Read:** Reading uncommitted data from another transaction.
- **Non-Repeatable Read:** Data changes between reads in the same transaction.
- **Phantom Read:** New rows appear in subsequent reads within the same transaction.

8. Savepoints

Definition

Savepoints allow partial rollback within a transaction, providing finer control.

Using Savepoints

```
START TRANSACTION;

INSERT INTO orders VALUES (101, 'Item1', 5);

SAVEPOINT sp1;

UPDATE inventory SET quantity = quantity - 5 WHERE item_id = 'Item1';

-- Something wrong detected
ROLLBACK TO SAVEPOINT sp1;

COMMIT;
```

9. Example Use Case: Bank Transfer

Safe money transfer using transaction

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE account_id = 123;
UPDATE accounts SET balance = balance + 500 WHERE account_id = 456;

COMMIT;
```

If an error occurs (e.g., insufficient funds), use ROLLBACK to cancel changes.

10. Summary

Summary

- Transactions group multiple DB operations into a single unit with ACID properties.
- Use BEGIN, COMMIT, and ROLLBACK to control transactions.
- Isolation levels affect visibility of data changes among concurrent transactions.
- Savepoints help partial rollbacks within transactions.
- Proper transaction management ensures data consistency and reliability.